Can a Smaller Network Keep Its Promise?

A worked guide to certified reduction, robustness and the information needed for updates

Worked guide • revised 6 November 2026

Imagine two almost identical measurements arriving at a drone's controller. We want the resulting commands to remain close. Now suppose we make the controller's neural network smaller. Can we keep that guarantee? And if its weights later change, what information must we retain?

We can answer the first question on two published Alsomitra networks. Lean, a proof assistant that checks mathematical arguments, verifies a precise robustness requirement and its preservation by neuron removal. The later examples explain why updating a reduced network requires an additional argument.

1. The promise, stated explicitly

A **checkpoint** is a saved set of learned network parameters. Here the network takes six normalized real numbers and produces one scalar command. Normalized inputs lie between 0 and 1; they are not physical-unit readings. Write **N(x)** for the network's output at input vector x.

An exact condition that is true or false for given arguments is a **predicate**. The input predicate **P** says that both inputs are in the permitted cube and that each of their six coordinates differs by at most 0.001:

$$P(x, y) \iff x, y \in [0, 1]^6 \ \wedge \ \forall i \in \{0, \ldots, 5\}, \ |x_i - y_i| \leq 0.001.$$

The output requirement **Q** says that the two commands differ by at most 0.1:

$$Q_N(x, y) \iff |N(x) - N(y)| \leq 0.1.$$

Lean proves this implication for each original and reduced DL2/Gödel network:

$$\forall x, y \in \mathbb{R}^6, \qquad P(x, y) \implies Q_N(x, y).$$

For example, take all six coordinates of x to be **0.5**, and all six coordinates of y to be **0.5005**. Their differences are 0.0005, so P holds. The theorem guarantees Q for this pair and for every other permitted pair.

Published checkpoint	Required maximum change	Proved maximum when P holds
DL2	0.1	0.031
Gödel	0.1	0.026

The names identify two archived model files. Their proofs cover all permitted input pairs, including pairs absent from the training data.

2. What does “every permitted pair” mean?

The six input coordinates represent normalized horizontal and vertical velocity, angular velocity, angle, and horizontal and vertical position. We use x and y for **whole input vectors**; a subscript such as x_4 selects one coordinate. All six coordinates can change together, independently.

Keep x fixed at (0.5, 0.5, 0.5, 0.5, 0.5, 0.5) and consider these choices:

Second input y	Does P hold?	What the displayed implication tells us
Every coordinate is 0.5005	Yes: all differences are 0.0005.	The output difference is at most 0.1.
(0.501, 0.499, 0.5, 0.501, 0.499, 0.5)	Yes: each difference is at most 0.001.	The same guarantee applies, with changes in several directions.
First coordinate 0.502; the rest 0.5	No: one difference is 0.002.	This P ⇒ Q contract makes no promise about this pair.

The example with 0.5005 also shows that the premise is meaningful: two different inputs strictly inside the cube satisfy it. Lean checks such a witness explicitly. This is called **nonvacuity**. It prevents a statement from appearing successful merely because its input condition is impossible.

Where the bound comes from

A network combines weighted sums with activation functions. A weight scales a signal; a bias adds a constant. In these models the hidden activation is **ReLU**, which replaces a negative number by zero and leaves a nonnegative number unchanged. ReLU cannot enlarge the distance between two scalar values.

The checker follows how input differences can grow through the weight matrices. It uses absolute weights, so positive and negative contributions cannot cancel and hide a possible large difference. This gives a conservative bound: it may be larger than necessary, but the proof establishes that it is safe to use.

For DL2, Lean accepts a multiplier of **31**. For Gödel it accepts **26**. Consequently, a coordinate perturbation of at most 0.001 gives:

$$31 \cdot 0.001 = 0.031 \leq 0.1, \qquad 26 \cdot 0.001 = 0.026 \leq 0.1.$$

The proof is derived from the exact saved weights. It does not assume that training succeeded or infer success from a sample of predictions.

This is **robustness of the output to nearby inputs**. It does not by itself tell us whether the command is the right one for flying the drone. A constant network would also be insensitive to perturbations. The saved models here are nonconstant; evaluating control quality remains a separate task from bounding sensitivity.

3. Removing a neuron without changing the answer

A hidden neuron first computes a weighted sum plus a bias. Call this number its **preactivation**. ReLU then produces the neuron's activation. If we prove that the preactivation is never positive on the whole input domain, this neuron always contributes zero.

For **both DL2 and Gödel**, the first hidden layer's neuron 58, counting from zero, has the following checked upper bound:

$$z_{58}(x) \le \frac{-9057167}{134217728} < 0 \qquad (x \in [0,1]^6).$$

The upper bound is approximately −0.06748, so its activation is always zero. Whether an outgoing weight is 7, −4 or a very small fraction, its contribution is that weight multiplied by zero. Removing the neuron and its outgoing terms leaves the later weighted sums unchanged.

An **interval bound** encloses all possible values between a lower and an upper limit. The **pruning certificate** contains these bounds and a **mask**, a keep/delete flag for each neuron. A certificate is data to be checked, rather than a promise we accept on trust. Lean proves that if these checks pass, the deletion preserves the network's mathematical output throughout the cube:

$$\forall z \in [0,1]^6, \qquad N_{\text{small}}(z) = N_{\text{original}}(z).$$

For each of the two checkpoints	Before	After
Layer widths: inputs, hidden layers, output	6 → 64 → 32 → 1	6 → 63 → 32 → 1
Stored weights and biases	2,561	2,522

All six inputs remain. We remove a hidden computation, not an input coordinate. The saving is modest: 39 parameters, about 1.52%.

Why the smaller network inherits P ⇒ Q

1. **Prove the original promise**
   Every pair satisfying P has output difference at most 0.1.

↓

2. **Prove exact reduction**
   The smaller network agrees with the original at every input in the cube.

↓

3. **Substitute at both inputs**
   Replace N(x) and N(y) by the smaller network's equal outputs.

Therefore the smaller network satisfies the same implication, including the stronger 0.031 and 0.026 bounds. There is no new search over input pairs at this transfer step: equality transports the existing guarantee.

4. Two certificates answer different questions

The preceding result needs two independent pieces of evidence. One says that the original network satisfies the promise. The other says that making it smaller preserves its outputs.

Certificate	What it checks	What it allows us to conclude
Sensitivity certificate	All weight matrices and a proposed sensitivity bound.	Nearby inputs have sufficiently close outputs.
Pruning certificate	Hidden-layer bounds and the selected deletions.	The smaller and original networks have equal outputs on the domain.

This distinction becomes important when weights change. The pruning certificate does not inspect the output weights: zero hidden activations still contribute zero after those weights change. The sensitivity certificate does inspect them, because larger weights can amplify input differences.

A small arithmetic example

Consider an illustrative network with output $\mathbf{N(x) = x_0}$, plus an unused hidden neuron whose activation is always zero. A second version has output $\mathbf{M(x) = 200x_0}$, with the same unused neuron.

Take $x_0 = 0.5$ and $y_0 = 0.501$, keeping the other coordinates equal:

Network	Output difference	Meets the 0.1 requirement for this pair?
N	0.001	Yes
M	0.2	No

The zero neuron can be deleted in both cases. Deletion alone therefore cannot establish robustness after an arbitrary weight update. These two simple functions illustrate the logic; they are not the saved checkpoints.

Exact preservation also preserves a failure

Our earlier experiment used two other published checkpoints, called **Baseline** and **Adversarial**. They have smaller architectures than DL2 and Gödel. On these models, Lean establishes counterexamples to two directional requirements at margin 2. A **refutation** is an exact input where the premise holds but the requested output condition fails.

For example, a Baseline input strictly inside the cube has physical position sum $X + Y \approx 3.518069$. The selected requirement says that when $X + Y \geq 2$ the normalized output must be at least 0.5. Its output is approximately 0.399148. Lean checks the exact fractions behind these rounded figures. Its reduced network gives the same answer and hence the same counterexample. Appendix A gives the complete input.

Exact reduction faithfully preserves the behavior we have. Repairing a failed requirement would be a further change requiring its own proof.

5. What are the state, the view and RR?

We now ask a different question: can a reduced representation support future parameter changes? To keep every step visible, this part uses the **Adversarial model's certified hidden layers**, held fixed. It is a separate example from the positive DL2/Gödel property above.

Its second hidden layer has activations ($h_0(x)$, 0, $h_2(x)$, 0) on the cube. Here $h_0(x)$ and $h_2(x)$ are the surviving neurons' values, computed from x by the fixed hidden layers. The **output head** is the last layer: four weights and a bias, written (a_0, a_1, a_2, a_3, b). Thus its output is:

$$N_a(x) = \mathrm{ReLU}\,(a_0 h_0(x) + a_2 h_2(x) + b).$$

For this example a **state s** consists of an input x in the cube and that five-parameter head. The hidden layers are fixed background data. The **view q(s)** keeps x and the three parameters needed for evaluation:

$$s = (x, a_0, a_1, a_2, a_3, b), \qquad q(s) = (x, a_0, a_2, b).$$

Two different states, the same retained information

Fix any one input x in the cube. Consider these illustrative heads; the general Lean theorem allows every rational head (parameters that are ratios of integers), including these:

State	Full head (a_0, a_1, a_2, a_3, b)	Retained head (a_0, a_2, b)
s	(2, 7, 3, 11, 1)	(2, 3, 1)
t	(2, −4, 3, 99, 1)	(2, 3, 1)

Both evaluate as $\mathrm{ReLU}(2h_0(x) + 3h_2(x) + 1)$. The different values 7 and −4 multiply zero; so do 11 and 99. The states differ, but the view cannot distinguish them.

RR names precisely this relationship:

$$RR(s, t) \iff q(s) = q(t).$$

RR is an **equivalence relation**: every state has its own view (reflexivity); equal views agree in either order (symmetry); and if two equalities share a middle view, the outer views agree (transitivity). Grouping states by their common view forms a **quotient**: each group retains one description of the information we chose to keep.

RR is stronger than “the outputs happened to match once.” It requires equality of the retained input and parameters. It also differs from P: **P compares nearby input vectors; RR compares full states with identical views.** This RR is defined on one state space, not between different types.

6. Can we still perform the next update?

Return to states s and t from the previous page. Both views contain the same head (2, 3, 1). Suppose we increase a_0 by 0.5. Both full heads then have $a_0 = 2.5$, and the reduced representation can perform exactly that update using information it already has.

Now try a different rule: **replace a_0 by $a_0 + a_1$**.

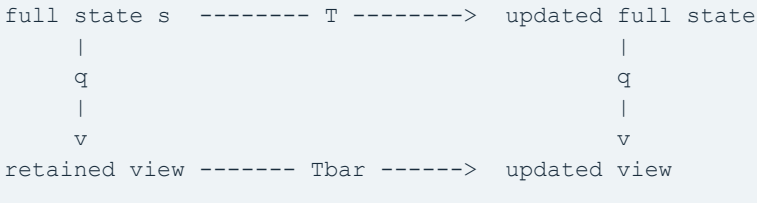
State	Old a_0	Forgotten a_1	New a_0
s	2	7	9
t	2	−4	−2

The reduced representation starts from (2, 3, 1) in both cases. It cannot deterministically choose both 9 and −2 as its new first coordinate. Information required by the update has been discarded. Lean proves this impossibility using the same pattern, with even smaller numeric witnesses.

The hidden neurons still contribute zero, so their pruning certificate remains valid. What fails is the ability to update using only the view.

The condition that makes an update compatible

Let **T** be an update on full states and **$\overline{T}$** its proposed reduced implementation. Compatibility means that these two routes agree:



$$q(T(s)) = \overline{T}(q(s)).$$

This equality is also called **factorization through the view**: the retained part of the result can be computed from retained information. If each permitted step has this property, **induction** gives it for every finite sequence: agreement holds before any steps; one compatible step preserves it; the argument repeats. Full and reduced execution stay in agreement. RR-related full states also remain RR-related under the same sequence. Choosing the next operation from the state must also use retained information.

An automatic dependency analysis can enlarge the view when necessary:

Permitted rules	Head coordinates that must be retained
Shared additive shifts	a_0, a_2, b: three coordinates
Also allow $a_0 := a_0 + a_1$	Add a_1: four coordinates
Also allow $a_1 := a_1 + a_3$	Add a_3: all five coordinates

Keeping an extra parameter for an update does not require restoring its zero hidden neuron. Parameter memory and inference computation serve different purposes.

7. Why does a subalgebra appear here?

The view tells us what information survives. We can also ask which quantities remain determined by that information. For the view q(s) = (x, a_0, a_2, b), consider:

Quantity we want to know	Can the view determine it?	Reason
a_0, or the sum $a_0 + a_2$	Yes	Both coordinates are retained.
The network output at the retained input	Yes	The deleted activations are zero.
a_1	No	The same view occurs with $a_1 = 7$ and $a_1 = -4$.
The value of a_0 after $a_0 := a_0 + a_1$	No	The rule reads information the view has forgotten.

A real-valued quantity attached to a state is called an **observable**. For example, $F(s) = a_0 + a_2$ is an observable. It factors through q because we can compute it from the view alone.

If two observables are determined by q, so are their sum and product, any constant multiple, and constant functions. That closure under arithmetic operations is why these observables form an **algebra**. They form a **subalgebra** of all real-valued functions on full states: some functions, such as the forgotten a_1, are excluded.

Formally, if V is the space of views, the subalgebra is:

$$\mathrm{Obs}(q) = \{f \circ q \mid f : V \longrightarrow \mathbb{R}\}.$$

Here composition means “first take the view, then apply f.” The condition is dependence only on the view; arbitrary real-valued functions are allowed, so this definition does not supply executable code for every member. Lean constructs an actual `Subalgebra` with this meaning.

A concrete calculation

For our shared view (x, 2, 3, 1), let $F(s) = a_0 + a_2$ and G(s) = b. We can recover **F = 5**, **G = 1**, **F + G = 6** and **FG = 5** from the view. Every state RR-related to s gives these same answers. We cannot recover a_1 by combining the retained quantities: s and t already show that it is not determined by them.

What “stable” means

After the compatible update $a_0 := a_0 + 0.5$, F becomes **5.5**. Its value changed, but it is still computable from retained information. A **stable subalgebra** keeps dependence on retained information after every allowed update. For any observable H in it, H after an update is still a function of the old view, via the reduced update.

That is the connection: **RR groups indistinguishable states; the subalgebra describes the quantities they agree on; compatible updates preserve our ability to calculate those quantities.** Stability does not say that outputs stay constant or that an earlier robustness bound automatically remains true.

8. Learning adds memory to the state

An **optimizer** is a rule for changing parameters to reduce a loss, which is a numerical measure of error or another chosen objective. A **gradient** describes how the loss changes with each parameter. Many optimizers also keep memory of previous steps. A state sufficient for inference may therefore be insufficient for continued learning.

For **momentum**, let a be one parameter and v its stored update memory. With current gradient g, momentum factor μ and step size η, our convention is:

$$v' = \mu v + g, \qquad a' = a - \eta v'.$$

As a simple arithmetic illustration, set a = 2, g = 0.2, μ = 0.9 and η = 0.01. If v = 0, the next a is **1.998**. If v = 1, it is **1.989**. The same current parameter and gradient do not determine the next parameter unless the memory is also known.

Sometimes a sum is enough

The checked Adversarial-head experiment has five parameters and five momentum values: ten scalar coordinates. We specify an update with an extra loss term coupling a_0, a_1 and a_3 through their sum. This is a chosen experiment, not a reconstruction of the authors' training rule.

Individual coordinates would force us to keep all ten. A **linear observation** is a sum or weighted sum of coordinates. Such observations can instead retain these eight quantities:

$$q(s) = (a_0, a_1 + a_3, a_2, b,\ \ v_0, v_1 + v_3, v_2, v_b).$$

For example, (a_1, a_3) = (7, 11) and (8, 10) both give the retained sum **18**. Likewise, momentum pairs (4, 6) and (3, 7) both give **10**. With the other coordinates equal, the eight-value view cannot distinguish these full states, and the specified updates do not need to.

An exact calculation produces a candidate view and reduced update rule. Lean then checks that they agree with full execution. For the concrete coupled instance it also proves that eight is minimal among invariant linear observation spaces retaining the required output coordinates. Here **invariant** means the space remains sufficient after each allowed update: no new independent observation is needed to determine the next view. This is a precisely delimited minimum, not a claim about every possible encoding. With no coupling, the same comparison needs only six values.

The reduced rule must account for aggregation

If a_1 and a_3 each decrease by 1, their sum decreases by **2**. Updating their stored sum by only 1 would be wrong. The formal transport result includes this factor of two; a checked small example exposes the error when it is omitted. The benefit is an executable smaller state with a justified update rule, rather than merely fewer names on a page.

9. RMSProp shows why the allowed states matter

RMSProp, short for Root Mean Square Propagation, uses a running average of squared gradients to scale each parameter's step. A parameter with a different history can move differently even when its present gradient matches another's.

For a separate exact-real example, we use the following specified rule. The history r is nonnegative, the decay is 1/2, the step size is 1, and the stabilizing constant 1 is outside the square root:

$$r_i' = \frac{r_i + g_i^2}{2}, \qquad a_i' = a_i - \frac{g_i}{\sqrt{r_i'} + 1}.$$

Can we keep only the sum of two histories, as we kept sums for momentum? The full state now contains the five head parameters and five histories. The proposed view is (a_0, a_1+a_3, a_2, b, r_0, r_1+r_3, r_2, r_b): it keeps the head observations and replaces momentum by history observations. Start with the head (1, 0, 0, 0, 0). The specified coupled loss gives $g_1 = g_3 = 1$. All other histories are zero. Compare:

Initial histories (r_1, r_3)	Retained sum	Histories after the step	New weight sum $a_1' + a_3'$
(1, 97)	98	(1, 49)	−1/2 − 1/8 = −5/8
(49, 49)	98	(25, 25)	−1/6 − 1/6 = −1/3

The old view is identical, but the new weight sums differ. The separate square roots make the distribution of history matter. Lean proves that no deterministic update on this view works for **all nonnegative histories**.

A stronger assumption makes the same view sufficient

Suppose the paired histories start equal. In this model their gradients always agree, so their new histories also agree. This is an **invariant**: a condition true initially and preserved by every permitted step. Zero initialization satisfies it.

If $r_1 = r_3$ and their stored sum is R, each history must be R/2. Nothing needed by the square roots is missing. With $u = a_1 + a_3$ and their common gradient g, the exact reduced rule becomes:

$$R' = \frac{R + 2g^2}{2}, \qquad u' = u - \frac{2g}{\sqrt{R'/2} + 1}.$$

Lean proves this rule correct on the invariant equal-history states, including every finite permitted input sequence. RR and the stable subalgebra are defined on that restricted state space. The failing (1, 97) witness cannot occur from equal histories under these rules.

The negative and positive results fit together: the sum fails on the larger state space, but succeeds on a justified smaller one. This real RMSProp model has not yet been connected to evaluation of the concrete rational-head checkpoints; the momentum example has that connection.

10. How the pieces support the controller project

The original research connects logical requirements with machine learning. Our fixed-checkpoint result adds a concrete chain after training:

1. **Published trained network**
   Bind the saved parameters to a precise mathematical network.

↓

2. **Proved requirement**
   Establish P implies Q for all permitted input pairs.

↓

3. **Certified smaller network**
   Remove inactive computation and transfer the same requirement.

The update examples ask what happens next. They show how to retain enough information for a smaller representation to evolve like the full one, and how to detect a proposed reduction that forgets too much.

These are connected questions, but the proofs apply to different objects:

Demonstration	What is established
DL2 and Gödel checkpoints	A true P ⇒ Q property; exact reduction; the same property for both smaller networks.
Baseline and Adversarial checkpoints	Exact reduction; exact refutations of selected directional requirements, preserved by reduction.
Adversarial hidden layers with specified head/momentum updates	Full and reduced networks agree after every permitted finite update sequence.
Separate real RMSProp model	A failing aggregate on general histories; a correct one on equal-history invariant states.

In particular, the optimizer results do not yet prove that the DL2/Gödel robustness property survives training. Full and reduced networks could agree after an update while both violate their former bound, as the simple scaling example showed.

A concrete next question for collaboration

Can we accept a useful parameter update only when its new checkpoint has a checked robustness certificate, while retaining the certified reduction? A candidate workflow would propose an update, check the new property, and accept only certified candidates. To develop this into a result, we would need an explicit update policy, a proved acceptance rule, and evidence that useful updates pass it. This is a proposed next experiment, not an existing theorem in the repository.

The present study gives that discussion concrete starting points: saved models, explicit requirements, checked reductions, successful and failing examples, and commands for replaying the evidence. The practical question is which requirements, updates and operating domain would make the next example useful for the authors' controller work.

11. What the proof says about a running program

The proofs interpret the saved weights exactly and perform network arithmetic over the real numbers. Each finite float32 checkpoint constant has an exact rational value; that value is retained in the mathematical model. A **rational** is a ratio of integers. The input values themselves range over reals, so the theorem is not restricted to finitely many representable inputs.

Actual float32 computation rounds intermediate results. Changing matrix shape can change the order of additions and hence the rounding. Exact real equality alone does not imply identical floating-point execution.

An observed difference in the last bit

In an earlier Baseline comparison, one input produced:

```
original output: 0.9654448628425598
reduced output:  0.965444803237915
```

These adjacent float32 values illustrate the distinction. Across 8,616 tested inputs, Baseline PyTorch batch-one execution differed on 337; the largest difference was approximately 0.000000477. Other tested execution layouts were bit-identical on that corpus. The real theorem does not claim to settle these runtime differences.

For DL2 and Gödel, original and reduced ONNX models were bit-identical on **4,520 inputs per model** in both tested layouts: single-input and whole-corpus execution. That corpus contains 360 published data rows, 64 cube corners and 2,048 pairs of nearby inputs. These observations support the implementation check; a universal float32 theorem would require a further numerical argument.

What has and has not been measured

The saved models were not retrained for this study. Their predictions are nonconstant. On the 360 published rows, their descriptive mean squared errors (the average squared difference from the stored targets) were approximately 0.025110 for DL2 and 0.033898 for Gödel, compared with 0.081213 for the best constant on those same rows. It is unknown which rows were held aside from these checkpoints' training, so these figures do not establish performance on unseen data or acceptable flight performance.

The parameter counts describe structural savings. They do not by themselves promise lower latency on a particular processor. A controller's flight behavior also depends on sensing, normalization, actuator dynamics and the physical system, which are outside the network theorem.

This separates three kinds of evidence a collaborator can examine: **mathematical guarantees**, **finite execution checks**, and **the remaining application evaluation**. The appendices locate each artifact without requiring the reader to work through every proof file first.

Appendix A. Exact models and reproducible examples

All four source checkpoints come from the authors' Alsomitra artifact repository, at revision `e9f6f1c52e0e5349455d2b4f69c2015567d3037f`. DL2/Gödel are under `vehicle_verification/property_verification_2/`; Baseline/Adversarial are under `python_training/models/`.

Checkpoint	Original → reduced widths	Parameters	Output activation
DL2 / Gödel, each	6-64-32-1 → 6-63-32-1	2,561 → 2,522	Linear
Baseline	6-6-4-1 → 6-5-4-1	75 → 64	ReLU
Adversarial	6-6-4-1 → 6-4-2-1	75 → 41	ReLU

The larger models have two hidden ReLUs and no final ReLU. The smaller pair has a final ReLU as well. The preservation proofs respect that difference. The original files remain unchanged; reduced exports are reloaded and their parameter bits bound to the formal reduced networks.

The exact bound behind the positive property

For weight matrices W_1, W_2, W_3, propagate the vector of initial unit bounds through their elementwise absolute values:

$$c_0 = (1, 1, 1, 1, 1, 1), \quad c_1 = |W_1|c_0, \quad c_2 = |W_2|c_1, \quad c_3 = |W_3|c_2.$$

The scalar result is approximately 30.332955 for DL2 and 25.385532 for Gödel; the exact checks accept 31 and 26. This is a standard conservative method, not an optimality claim. Biases cancel when comparing outputs of the same affine layer at two inputs.

The companion paper's Section 7.2 and Vehicle `LipschitzRobustness` use an output threshold Lstar/epsilon. The selected published pair is epsilon = 0.001 and Lstar = 0.0001, giving 0.1. Lstar is **not** the proved Lipschitz multiplier 31 or 26. The paper's Table 2 evaluates different model variants; applying its threshold here does not reproduce one of its reported cells. The Vehicle-to-Lean correspondence is reviewed manually; no verified Vehicle compiler is claimed.

The directional counterexample from Section 4

Input order is [vx, vy, omega, theta, x, y]. Take the Baseline input [1/4, 1/2, 1/2, 1/4, 1/4, 3/4]. The physical position conversions are:

```
X = 41.232296222734604 * input[4] + 0.482420778264618
Y = 45.878638794593570 * input[5] - 41.681405266088724
```

Lean proves X + Y > 2 and output < 1/2 using exact arithmetic. The property-decision guide gives all four original-model witnesses and their transfers. These refutations are consistent with the companion paper's larger reported critical distances; they are not a claim of a newly discovered flight failure.

Appendix B. Details behind the update examples

Certificates, state and preservation

The pruning checker inspects intervals and keep masks. For a neuron z = sum($w_i x_i$) + b in an input box, it checks an upper bound obtained by summing each term's maximum. Propagating those bounds through ReLU encloses the next layer. A deleted neuron must have upper bound at most zero. This may miss removable neurons when it loses correlations, but an accepted deletion is sound.

The head-update example's state type includes the evaluation input. The momentum and RMSProp modules instead take evaluation features as external arguments. Their states contain parameters and optimizer memory. In each case q is defined on that stated space; RR always means equality of its views.

The specified coupled momentum rule

Let θ = (a_0,a_1,a_2,a_3,b), let z = (h_0,0,h_2,0,1) be frozen features, and let y be a target output. The chosen loss is:

$$L(\theta) = \frac{(\theta \cdot z - y)^2}{2} + \frac{\lambda}{2}(a_0 + a_1 + a_3)^2.$$

Its algebraically specified update direction is G = (θ·z − y)z + λ(a_0+a_1+a_3)(1,1,0,1,0). The steps are v′ = μv + G and θ′ = θ − ηv′. No automatic differentiation implementation is verified.

The concrete synthesis instance uses checkpoint-derived features from the first published training row, y = 3/4, η = 1/100, μ = 9/10 and λ = 1/100. A second instance has λ = 0. The general correspondence proof allows rational configurations to vary across a finite sequence. It compares both networks at the resulting current heads; it makes no claim that the original output guarantee or loss value stays unchanged.

An **affine update** multiplies a state by a matrix and adds a constant vector. For T(s) = As + c, a linear view q(s) = Bs is checked by BA = ĀB and Bc = c̄. These equations justify the reduced rule v ↦ Āv + c̄. Repeatedly adding the dependencies of required observations produces a closed row space. Each matrix row specifies one weighted sum; its row space contains all linear combinations of those sums. Closure means the updates require no observation outside that space. Lean checks the emitted derivations and the scoped eight-dimensional minimum.

For a head projection B and a direction of the form G(θ) = B^Tg(Bθ), the projected direction is BB^Tg, not generally just g. In this example BB^T = diag(1,2,1,1): the sum coordinate accounts for two full coordinates. This supplies the factor of two explained in Section 8.

The RMSProp invariant

The separate real model uses features (0,0,z_2,0,1), target zero and coupled penalty (a_0+a_1+a_3)²/2. Its paired gradients always agree. Consequently r_1' − r_3' = (r_1 − r_3)/2; equal nonnegative histories remain equal and nonnegative. The reduced rule is proved on this subtype, for finite sequences of real z_2. Square roots can yield irrational parameters, so this development does not silently reuse the concrete rational-head evaluation theorem.

Appendix C. Where the claims are checked

Lean's **kernel** checks that a proof establishes its stated theorem. The theorem's definitions and assumptions still determine its meaning. The local checkers reject invalid certificates; the Python programs that propose bounds or reductions are not trusted to establish a theorem.

Claim in this guide	File and named result
Exact P/Q meaning and nonempty premise	Lipschitz.lean: `Property5`, `property5_two_sided`, `nonvacuous`.
Accepted sensitivity bound implies the property	Same file: `checked_global`, `property5_of_check`.
Positive checkpoint and reduced-model implications	DL2Checkpoint, GodelCheckpoint: `property5`, `same_output`, `reduced_property5`.
Exact pruning with a linear output	LinearPruning: `certified_linear_prune`.
RR, compatible updates, subalgebra and failure	OutputUpdates: `stateSetoid`, `run_factors`, `view_algebra_stable`, `bad_no_reduced_step`.
Momentum correspondence and retained-state synthesis	MomentumQuotient, LinearObservations; guide.
RMSProp failure and invariant-based success	AdaptiveBoundary, SymmetricRMSProp.
Property and counterexample transfer	PropertyTransfer, Witnesses.

The previously recorded audits cover the following roots. Counts overlap; they should not be added to claim a number of distinct new theorems.

Recorded development	Audit roots and verification receipt
Checkpoint pruning	43 roots
Output-head updates	36 roots
Coordinate retention	78 roots
Linear observations	82 roots
RMSProp / property transfer	47 roots
Concrete property decisions	48 roots
Positive property and its reduction	53 roots

The accepted audits report only Lean's standard `propext`, `Classical.choice` and `Quot.sound` axioms. The positive-property run checks nine compilation stages with no errors or warnings. Negative controls reject a zero sensitivity bound, deletion of active neurons and a wrong ONNX transpose. There is no new second-kernel validation. For this shared snapshot, the same proof sources are rebuilt in an isolated checkout; the linked receipts record that replay. The mathematical statements are unchanged.

Appendix D. Try the positive example first

The shortest path through the artifacts is the positive property on DL2/Gödel. Read its focused guide, then run the following command from this repository's root:

```
python -B contribution/network_reduction/positive_property/verify.py \
  --upstream /path/to/SAIV_2025_Alsomitra
```

Replace the path with a checkout at the revision in Appendix A. Use a Python environment with NumPy, ONNX and ONNX Runtime. The runner uses the repository's pinned Lean/mathlib installation and validates the six earlier PASS receipts and their proof-source bindings. It rebuilds the needed local proof modules, regenerates both reduced ONNX exports and runs the finite execution comparisons. The focused guide documents setup and artifact locations; these commands are not a standalone installation of Lean or the runtime packages.

The resulting positive-property receipt records the source hashes, compilation results, checked theorem roots, invalid-certificate controls and finite runtime evidence. A PASS means those checks succeeded. For the separate refutation experiment, PASS means the counterexample was verified, not that the refuted controller property was true.

For other parts, the guides provide the exact state definitions and runners:

- Fixed-checkpoint reduction and head updates.
- Coordinate retention and linear observations.
- Equal-history RMSProp and property transfer.
- Directional property decisions.

The synthesis stages validate predecessor bindings. When replaying their full chain after a document revision, regenerate predecessors in order rather than treating an expected old-document hash mismatch as a failed mathematical theorem. The repository README retains the ordered commands.

Sources and relation to the original work

- A General Framework for Property-Driven Machine Learning, Thomas Flinkow et al., 2025, version 2: the originating methodology.
- Neural Network Verification for Gliding Drone Control: A Case Study, Colin Kessler et al., 2025: controller requirements and companion experiments, particularly Section 7.2.
- Pinned Alsomitra repository: the exact model and specification artifacts used here.

The checkpoint-specific training runs are not reproduced. Exact reduction, linear observation spaces and equality-based property transfer are existing mathematical ideas; no priority claim is made for them. The inspectable result here is their checked application to the stated artifacts and update rules, with examples showing both where the reasoning succeeds and where an extra assumption is necessary.

To rebuild this PDF, install ReportLab, markdown-it-py, Matplotlib and svglib in the document environment and use the macOS fonts named by the shared renderer, then run:

```
python contribution/network_reduction/build_report.py
```